

Special Section on SMI 2024

Dynamics simulation-based packing of irregular 3D objects

Qiubing Zhuang^a, Zhonggui Chen^{a,*}, Keyu He^a, Juan Cao^a, Wenping Wang^b^a Xiamen University, China^b Texas A&M University, United States of America

ARTICLE INFO

Keywords:

Geometry processing
3D irregular packing
Dynamics simulation

ABSTRACT

The 3D packing problem has a wide range of applications. However, the complex geometry of irregular objects leads to a sharp increase in the number of placement combinations, making it a challenging problem. In this paper, we propose a packing pipeline based on rigid body dynamics simulation to deal with two types of 3D packing problems. One is the variant bin packing problem, which involves placing more objects into a container of given dimensions to maximize space utilization. The other is the open dimension problem, where the goal is to minimize the container that can accommodate all objects. We first use heuristic placement strategies and a fast collision detection algorithm to efficiently obtain initial packing results. Then, we simulate the shaking of the container according to the dynamic principle. Combined with the vacant space filling operation, shaking the container drives the movement of objects in the container to make the arrangement of objects more compact. For the open dimension packing, the container height is optimized by adjusting the constraints of simulation in the basic pipeline. Experimental results show that our method has advantages over existing methods in both speed and packing density.

1. Introduction

The 3D packing problem aims to place a set of objects into a container without overlapping to maximize space utilization. The packing of regular 3D objects and simple polyhedron [1] has been extensively studied in the past decades [2,3]. Due to its wide applications in logistics and manufacturing, 3D shape packing has received a great deal of research interest in the field of robotics [4–8] and computer graphics [9–11]. In robotics, researchers are exploring the use of robotic arms to automate the packaging of goods. They need to consider both the feasibility of robotic arm movements and packing strategies to maximize space utilization. In addition to maximizing space utilization, selecting the appropriate size of cartons is crucial when packing goods, as it significantly contributes to saving materials and reducing transportation costs. In the online packing problem [7], the logistics robot loads the items arriving in sequence into the target bin without knowing the future order of the items, and positions of the items are not allowed to adjust later. In 3D printing, graphics researchers aim to minimize the bounding box volume of the packed segments to save printing time. Compared with 3D regular objects and simple polyhedrons, general 3D irregular objects have more complex geometry and more rotational degrees of freedom, making the problem more challenging. Currently, research on 3D irregular shape packing has made some progress, but the utilization of space and efficiency remain

suboptimal. Furthermore, existing studies only consider the packing problem of rigid bodies and do not include many deformable objects, such as plush toys, fabrics, etc.

In this paper, we focus on two types of packing problems. Different from the online packing problem, the types and quantities of all objects to be packed are predetermined before packing. One is to place a group of 3D irregular objects into a target-sized container to maximize packing density. Packing density is defined as the sum of the volumes of all packed objects divided by the volume of the container. We aim to pick packing objects from these and place as many objects as possible into the container. We also consider the open dimension problem. In this case, the types and quantities of packed objects are given before packing. We fix the length and width of the container and then minimize the height of the container to maximize the packing density. After packing, the dimensions of the container can be referenced to select the most suitable cardboard boxes for the actual packing of the goods [12]. When there are soft bodies in the goods, the influence of deformable objects on the packing results should not be ignored during packing.

In recent years, research on the packing problem of 3D irregular objects has made some progress. Lamas et al. [13] adopted voxels to represent the geometry of objects and proposed no-fit voxels (NFV) to check whether objects intersect. Cui et al. [11] used the Fast Fourier Transform (FFT) to efficiently calculate the correlation

* Correspondence to: School of Informatics Xiamen University, Xiamen, China.

E-mail addresses: zhuangqiubing@163.com (Q. Zhuang), chenzhonggui@xmu.edu.cn (Z. Chen), 1205486810@qq.com (K. He), Juancao@xmu.edu.cn (J. Cao), wenping@tamu.edu (W. Wang).

<https://doi.org/10.1016/j.cag.2024.103996>

Received 20 May 2024; Received in revised form 30 June 2024; Accepted 4 July 2024

Available online 6 July 2024

0097-8493/© 2024 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

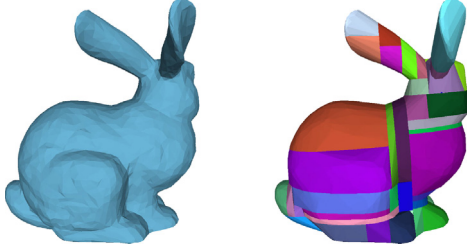


Fig. 1. Left: The origin shape. Right: A combination of convex shapes as an approximate representation of the original one.

between discrete objects as a collision metric. However, they limit the rotation of objects to discrete directions, which is not conducive to improving packing density. Ma et al. [14] and Romanova et al. [1] allow objects to rotate freely during packing, but this requires a time-consuming optimization process. We propose a method based on rigid body dynamics simulation to balance packing density and efficiency. The key insight is drawn from granular physics research, where it is observed that shaking the container horizontally periodically can cause interactions and movements among the particles, leading to compact arrangements [15–17], known as self-assembly [18,19]. In the physical simulator, GJK [20] algorithms and axis-aligned bounding box trees are usually used to implement real-time collision detection for convex shapes. For nonconvex irregular objects, they can be decomposed into multiple convex components [21] for physical simulation, as shown in Fig. 1.

In the framework of our algorithm, we use the Deepest-Bottom-Left-Fill (DBLF) heuristic strategy [22] to efficiently place objects in the container as the initial packing result. During the initialization, the objects are voxelized, and the FFT is used to efficiently calculate the collision metric between the objects. We demonstrate that periodically shaking the container can induce the rearrangement of different kinds of irregular objects during simulation. Finally, we search for empty spaces in the container and insert objects to fill the container.

Based on the above framework, we address the open dimension problem by simulating compression. Additionally, we integrate a finite element method to extend our method to soft body packing. To evaluate the performance of our method, we extensively test it on various types of data. Experimental results show that our method achieves a higher packing density because the position and rotation of the packing object are always allowed to be adjusted continuously during the packing process. Overall, our contributions include:

- An efficient initialization of packing. Combined with the collision metric in the spectral domain calculated by FFT, the DBLF heuristic strategy is used in initialization to realize efficient empty space finding and compact object placement.
- An effective packing optimization method based on rigid body dynamics simulation, including shaking the container and an empty space filling strategy.
- A flexible packing framework based on physical simulation. The packing framework can solve a variety of packing problems, including the 3D variant bin packing problem, open dimension problem, and soft body packing.

2. Related work

The packing problem aims to maximize packing density while ensuring there is no overlap between objects. 3D packing problem has a wide range of applications, and there are different variations in different applications. Among them, the most basic are 3D bin packing problem and 3D open dimension problem, also known as the strip

packing problem [23]. Here, we only review the most relevant packing problems and methods for 3D objects.

Heuristic Methods. Previous approaches to cargo packing have predominantly relied on heuristic strategies. Wang et al. [22] employed the DBLF heuristic strategy, which sequentially places objects in the deepest, bottom-most, and left-most positions. The DBLF heuristic strategy is prone to creating holes, leading to space waste. Wang and Hauser [24] improved the limitation of DBLF by introducing a heuristic strategy that reduces unfilled gaps through heightmap minimization. However, heightmap minimization ignores the difference between different positions at the same level. Heuristic methods struggle to achieve a high packing density when dealing with complex 3D irregular objects.

Learning-based Methods. Recently, Hu et al. [12] applied deep reinforcement learning (DRL) to solve the packing problem. To reduce the cost of packaging, they used DRL to optimize the packing order of a series of cuboids so that the surface area of the container was minimized. Similarly, we try to minimize the volume that can hold all the irregular objects. In the transport-and-pack (TAP) problem, TAP-Net [5] deals with both finding the optimal transport sequence and optimizing the rotation of the cuboid. Zhao et al. [25], considering the given packing order and physical stability constraints, trained agents to learn packing policy to deal with online cuboid packing problems. Furthermore, they proposed the packing configuration tree [8], which fully describes the state and action space, facilitating effective training and performance of the DRL agent. Their latest work [6] proposed a pipeline to learn online packing skills and object-ordering policy for irregular 3D shapes. Some learning-based methods utilized physical simulation for training, whereas our work focuses on optimizing the placement and rotation of packing objects using physical simulation.

Irregular Objects Packing. The first challenge for packing 3D irregular objects is to deal with collision detection. Different geometric representations of objects require different methods for collision detection and packing optimization. Romanova et al. [1] proposed quasi phi functions for simple concave polyhedrons to describe non-overlapping and distance constraints, and then convert the packing problem to a nonlinear optimization problem. This method allows the objects to rotate and shift continuously, but it takes a long time to obtain a local optimum. For the packing of 3D irregular objects, Ma et al. [14] optimized the orientation and position of the objects from their initial placement, and then combined the combinatorial optimization method to reduce the gaps between the objects. When minimizing the height of a container, it requires determining the optimal height through binary search and running the algorithm repeatedly, which is very time-consuming. Lamas et al. [13] used voxels as the geometric representation of irregular objects and designed two meta-heuristic algorithms for optimization based on the voxel representation. To reduce the supporting materials, build time, and assembly costs of 3D printing, Chen et al. [10] coarsely decomposed the input object into multiple small parts. The parts are then voxelized and packed, with the goal of minimizing the maximum height of the pile in the container. Zhao et al. [6] decomposed the objects into convex shapes for efficient collision detection in physical simulations. The same strategy is also used in our method. Cui et al. [11] used the voxel representation and FFT for discrete placement search. They performed efficient collision metric computation in the spectral domain. Then, adopting a greedy strategy, objects are sorted by volume from largest to smallest and packed sequentially. At each placement, the goal is to minimize the distance between the new object and the objects already in the container, resulting in a compact arrangement. Our empty space search uses the same approach to compute collision metric using FFT, but with a different placement strategy.

3. Problem description

The variant bin packing problem. The given container C takes up the space $[0, S_x] \times [0, S_y] \times [0, S_z] \subset \mathbb{R}^3$ and a set of n objects denoted by $G = \{G_1, \dots, G_n\}$ are all represented as closed manifold triangle

meshes. For each G_i , we use approximate convex decomposition [21] to decompose it into almost convex parts. The convex hulls of these almost convex parts are represented by triangle meshes, $D_i = \{d_{i1}, \dots, d_{ik_i}\}$. The combination of convex meshes can be used for efficient collision detection in physical simulation and represent the original meshes approximately as shown in Fig. 1. We treat the combination of convex meshes as a whole shape and call it an approximate shape D_i of G_i . Thus, the set of shapes to be packed is $D = \{D_1, D_2, \dots, D_n\}$. Let $V(\cdot)$ denote the volume function, and the packing density is computed as $\sum_{i=1}^m V(G_i)/V(C)$, where $m \leq n$ stands for the m selected packing objects. The goal is to maximize the packing density by selecting D_i from D and optimizing their arrangement.

The open dimension problem. Given a set of n objects denoted by $G = \{G_1, \dots, G_n\}$, they are all represented as closed manifold triangle meshes. For each G_i , its convex decomposed meshes are $D_i = \{d_{i1}, \dots, d_{ik_i}\}$, and $D = \{D_1, D_2, \dots, D_n\}$. Given the length and width of the container, the goal is to find the minimum height of the container such that it can accommodate all objects.

4. Method

Our framework consists of two parts: initialization and objects rearrangement based on dynamics simulation. We start by voxelizing approximate shapes and using DBLF heuristic algorithm to place them into the container. Subsequently, the physical simulation is performed to optimize the arrangement and make it more compact. Finally, we adapt our framework to different packing problems.

4.1. DBLF heuristic placement

During the initialization, we place objects into the container one by one. To make sure the new object does not overlap with objects in the container and the container itself, we voxelize both the container (including the objects inside) and the object to be placed. We use a collision metric [11] of voxel grids during initialization to avoid intersections. The collision metric between two voxel grids can be described as their correlation. The correlation in the spectral domain is computed using the FFT, with a time complexity of $O(n \log n)$ [11]. First, we divide the interior space of the container into a voxelized grid of dimensions $N_x \times N_y \times N_z$. Then we add an additional layer of boundary voxels around the container to act as its border, as shown by the green cells in Fig. 2(b). At this point, the voxelized grid size of the container is $(N_x + 2) \times (N_y + 2) \times (N_z + 2)$. Before packing, the value of voxels inside the container is 0, and the boundary voxel value is 1. Finally, the object A to be packed is voxelized within the same bounding box as shown by the orange cells in Fig. 2(a). Empty voxels are designated a value of 0, while occupied voxels are assigned a value of 1. The voxelized grids of object A and the container C (including the objects inside) are represented as follows:

$$\Omega_A(\mathbf{x}) = \begin{cases} 1 & \text{if voxel } \mathbf{x} \in A, \\ 0 & \text{if voxel } \mathbf{x} \notin A. \end{cases} \quad (1)$$

$$\Omega_C(\mathbf{x}) = \begin{cases} 1 & \text{if voxel } \mathbf{x} \in C, \\ 0 & \text{if voxel } \mathbf{x} \notin C. \end{cases} \quad (2)$$

where $\mathbf{x} = (i, j, k) \in \mathbb{Z}^3$ and $0 \leq i \leq N_x, 0 \leq j \leq N_y, 0 \leq k \leq N_z$.

The correlation $\zeta_{A,C}(\mathbf{q})$ between $\Omega_A(\mathbf{x})$ and $\Omega_C(\mathbf{x})$ represents the number of voxels A 's voxel grid overlapped with C 's voxel grid when object A has displacement $\mathbf{q} \in \mathbb{Z}^3$

$$\zeta_{A,C}(\mathbf{q}) = \sum_{\mathbf{x}} \Omega_A(\mathbf{x}) \Omega_C(\mathbf{x} - \mathbf{q}). \quad (3)$$

FFT and inverse FFT are used to efficiently compute $\zeta_{A,C}(\mathbf{q})$ in the spectral domain to quantify the collision between objects. For computational details, we refer the readers to [11]. The output of inverse FFT is the collision metric which records the number of voxels that overlap

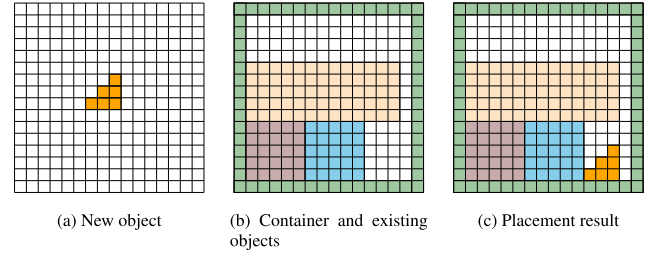


Fig. 2. Empty location searching and DBLF placement. Empty location searching can find holes inside the container. The DBLF heuristic method can fill these holes from bottom to top.

between the voxel grid Ω_A and the voxel grid Ω_C at each displacement $\mathbf{q}$.

When $\zeta_{A,C}(\mathbf{q}_0) = 0$, we refer to $\mathbf{q}_0$ as an empty location. It indicates that object A , when placed at displacement $\mathbf{q}_0$, will not intersect with the container (including any objects inside). We begin searching from the deepest, bottom-most, and left-most displacement until we find the first empty location and place the object A at that position. If intersections exist at all displacements, we adjust the orientation of object A and recompute the collision metric. We divide the Euler angles uniformly to sample the orientations. Allowing objects to have more orientations makes it possible to put more objects into the container during initialization, but it also takes more time. The subsequent physical simulation process will allow the object to rotate continuously, so in the initialization, we only consider sampling Euler angles at 90° along each coordinate axis. Regarding the order of objects to be packed, we sort the objects by volume from largest to smallest and then search and place them one by one. During the initialization phase, we try to place each object into the container until no more objects can be placed. As shown in Fig. 2, the DBLF heuristic placement can fill holes inside the container.

4.2. Physical simulation-based objects rearrangement

After initialization, objects have been placed in the container. Due to the discrete rotations allowed for objects and the imprecise voxel representation, there are gaps or empty spaces between objects. According to the findings of granular material research [16,18], that the horizontal periodic shaking of the container can make the objects in the container more compact, we use rigid body dynamics simulation to implement this procedure and achieve a more compact arrangement. We first introduce the process of object rearrangement when the container is shaken, and then implement this process using the impulse-based rigid body dynamics simulation method.

When shaking a container filled with granular material, the objects inside receive additional kinetic energy, prompting them to move or rotate. At the same time, the objects in the container are also subjected to the force of gravity, which causes them to move towards positions with lower gravitational potential energy. During the shaking process, the objects undergo inelastic collisions, experiencing continuous rearrangement and directional adjustments. Since inelastic collisions cause the kinetic energy of the objects to gradually decrease, the objects within will gradually stop moving once the shake of the container stops. When there are no other objects around an object, it has a large space to move. If there is no more space around the object, it will move only slightly. That is, compact arrangements will be more stable under horizontal shaking. When the non-compact part of the arrangement is filled by the moving object, a more compact arrangement is produced in this part. During the shaking process, the unstable arrangement is gradually transformed into a stable one, while the stable arrangement hardly changes. As a result, the original gaps between the objects can be effectively eliminated, leading to a more compact arrangement.

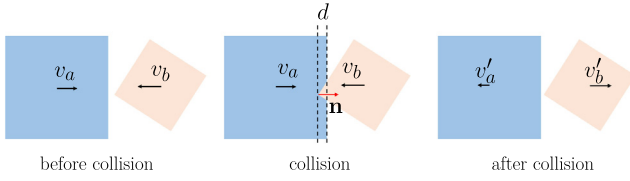


Fig. 3. Two rigid bodies collide and separate.

For physics simulation, we set the container to a static rigid body, which means that the container is not subject to external forces, and its position is completely controlled by us. In the case of 3D, the container is composed of six cuboids. In the 2D case, the container is formed by four rectangles, as shown in the green parts in Fig. 4. The objects inside the container are set as dynamic rigid bodies, and they will move and rotate when subjected to external forces. Rigid body dynamics simulation involves the computational simulation of the motion and interactions of rigid bodies.

Starting from the initial state and at each time step, the impulse-based method updates the velocities and positions of bodies by calculating and applying the impulses generated at the points of collision and contact between rigid bodies. For example, applying an impulse $\mathbf{J}$ on a rigid body with mass m results in a change in linear velocity $\Delta \mathbf{v} = \mathbf{J}/m$. The impulse also causes a change in the angular momentum of the rigid body, which causes the angular velocity of the colliding rigid body to change. In physical simulations, it is necessary to detect any collisions between all objects as well as between objects and the container. This includes calculating the location of the collision, the collision normal $\mathbf{n}$, and the penetration depth d . Fig. 3 illustrates the process of two rigid bodies colliding and separating. When a collision between two objects is detected, it is required to calculate the impulses applied to both objects based on collision information. Since friction can hinder the motion of objects, we do not consider friction during the simulation. To simulate frictionless and inelastic collisions, the impulses applied to both objects need to satisfy Newton's law of restitution and prevent the objects from continuing to penetrate each other. Finally, the physics simulator applies the calculated impulse to the objects, updating their velocities and positions to simulate the correct collision response.

We control the container to undergo periodic horizontal movements by applying the following equation:

$$x = \alpha \sin \omega t, \quad (4)$$

where x is the position of the container along the X -axis, α is the amplitude, and ω is the angular frequency. The process in which the container completes k periods of horizontal movement until the objects inside the container remain stationary is referred to as one oscillation. Fig. 4 illustrates the transition of objects within the container from dispersed arrangement to compact arrangement after multiple oscillations. Initially, the container and adjacent objects collide, causing the objects to gain kinetic energy. The objects, after gaining kinetic energy, move and collide with surrounding objects, leading to the rearrangement of

objects within the container. When the container stops moving, due to collisions between objects in the simulation following Newton's laws of motion, their velocities will decrease to zero.

4.3. Greedy objects insertion strategy

The oscillation of the container leads to a more compact arrangement of the objects inside. Therefore, more empty space can be free up to accommodate extra items. We sort the remaining unpacked items by volume from largest to smallest and greedily insert them into the container by calculating the collision metric. However, as long as there are gaps in the container that do not perfectly match the shape of the objects, the collision metric between the container and object will be greater than zero. In Section 4.1, we introduce the collision metric, which describes the number of intersecting voxels between the new object and the current state of the container. When the collision metric is small, it indicates that the shape of the object and the gap are almost matched. Only slight adjustments to the poses of the surrounding objects are needed to accommodate the current object.

To address this, we use the constraint solver of the physics simulator to simulate the process of the inserted object gradually pushing away the surrounding object. First, we shrink the current object and insert it into the empty location found based on the collision metric. Then, while shaking the container, we gradually enlarge the object to its original size. During the enlarging process, there may be small overlaps between the inserted object and surrounding objects. The physics simulator will attempt to find a set of impulses that can resolve all contact constraints, meaning there are no overlaps between any objects. This allows us to adjust the pose of other objects and insert new objects. If overlapping objects remain in the container due to unmet contact constraints or exceeding the maximum number of iterations, the inserted objects will be removed from the container. After packing the approximate shapes, they will be replaced by the original shape to obtain the final packing result. The original shape is decomposed into several almost convex parts, and the convex hulls of these parts form the approximate shape. Therefore, the approximate shape covers the original mesh and can still guarantee that there is no intersection between the original shapes after being replaced. Combining collision metric and scaling inserted objects, the greedy object insertion algorithm for the variant bin packing problem is illustrated in Algorithm 1.

4.4. Optimization of container size

For the open dimension problem, we initially place all given objects during the initialization and then optimize the height of the container using physical simulation. When inserting objects, we scale the inserted object. Similarly, when optimizing the container height while combining container shaking, we gradually reduce the container height.

After completing one oscillation, we obtain the height H of the highest vertex of the objects inside the container. At this point, the container only needs a height of H to accommodate all objects. Compared

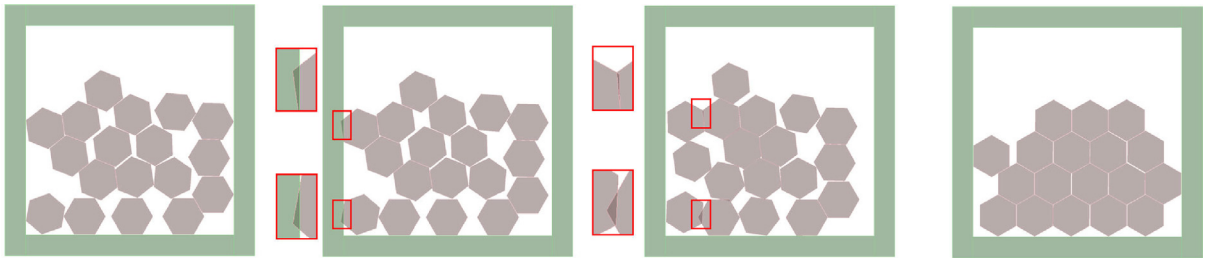


Fig. 4. Objects rearrangement. When the container starts to shake, it first collides with objects on the edge of the container, causing those objects to gain kinetic energy. These objects then move and collide with other objects, transferring their kinetic energy to nearby objects. After the container stops moving, the objects in the container gradually stop moving and a more compact arrangement is obtained.

Algorithm 1 Greedy Objects Insertion

Input: A list of objects A , a list of successfully packed objects P , a scaling factor s .

Output: A list of successfully packed objects P .

```

1:  $A \leftarrow \text{Sort}(A)$   $\triangleright$  Sort objects from largest to smallest
2: for each object  $o \in A$  do
3:    $\{b, t, r\} \leftarrow \text{InsertObject}(o, P)$ 
4:   if  $b$  then  $\triangleright$  Insertion successful
5:      $P.add(o)$ 
6:      $\text{AddRigidBody}(o, t, r)$   $\triangleright$  Add  $o$  to simulator
7:   else
8:      $o' \leftarrow \text{ScaleObject}(o, s)$ 
9:      $\{b, t, r\} \leftarrow \text{InsertObject}(o', P)$ 
10:    if  $b$  then  $\triangleright$  Insertion successful
11:       $\text{AddRigidBody}(o, t, r)$ 
12:       $\text{ShakeContainerAndEnlargeObjects}(o')$ 
13:      if  $\text{CheckOverlap}()$  then  $\triangleright$  Overlapping
14:         $\text{DeleteRigidBody}(o)$ 
15:      else
16:         $P.add(o)$ 
17:      end if
18:    end if
19:  end for
20: return  $P$ 

```

to the highest object, other objects still have some space from the top of the container. To further optimize the container height, we move the top cuboid of the container to a height of $H - \Delta h$. The top object will have a slight overlap with the container, causing it to move due to the impulse applied to it. If after constraint solving, there is no overlap between all objects, it indicates that the container can accommodate all objects at the current height and can continue shaking and reducing the container height. Otherwise, we need to adjust the container's height to H and shake the container again. The algorithm for optimizing the container size is described in Algorithm 2.

Algorithm 2 Optimization of container size

Input: A list of packed objects P , height decrement Δh

Output: The height of the container H that holds all the objects.

```

1:  $\text{ScaleContainer}(H)$   $\triangleright$  Reduced container height to  $H$ 
2: while true do
3:    $\text{ScaleContainer}(H - \Delta h)$ 
4:    $\text{ShakeContainer}()$ 
5:    $b \leftarrow \text{CheckOverlap}()$ 
6:   if  $b$  then  $\triangleright$  Overlapping
7:     return  $H$ 
8:   else
9:      $H \leftarrow H - \Delta h$ 
10:  end if
11: end while
12: return  $H$ 

```

5. Experiments

We implement our algorithm by using C++ and CUDA, and perform all experiments on a laptop equipped with an AMD Ryzen 7 6800H CPU and an RTX 3050 Ti Laptop GPU. We use cuFFT library [26] to calculate the collision metric. The smallest edge of the container is divided by 128 as the voxel size used for all 3D packing experiments. The container shakes with a period $T = 1$ seconds and amplitude $\alpha = 0.1S_x$. We also implement the packing algorithm of 2D irregular objects. In 2D packing, Box2D [27] is used for physics simulation, while in 3D packing, we implement the physics simulation based on PhysX [28].

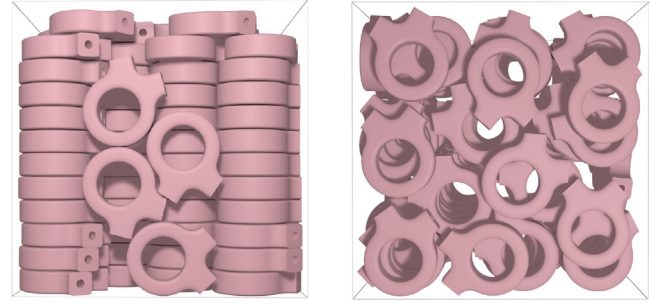


Fig. 5. Our algorithm loads 137 ring-shaped parts into a cube with a packing density of 43.19% within 132 s.

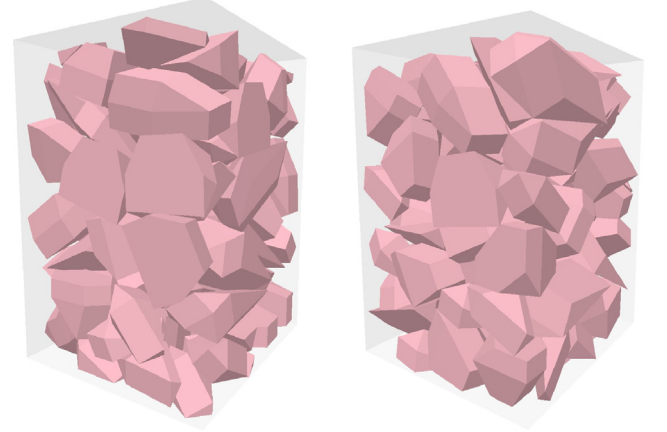


Fig. 6. Our packing results of Example 5 in [1]. **Left:** Our algorithm loads 86 polyhedrons into a cuboid with dimensions $36.98 \times 41.98 \times 56.37$. We achieve a packing density of 54.47% within 277 s. Cui et al. [11] load 80 polyhedrons into the same container, with a packing density of 51.27%, and 329 s. **Right:** We pack the same 80 polyhedrons and minimize the container height, with the packing density of 54.95%, and 284 s. With only 102 s, our algorithm achieves a packing density higher than 51.27%.

Since the packing density is closely related to the shape of the packing objects, we test our algorithm on a dataset of various shapes. Previous 3D irregular packing algorithms can achieve good packing density in irregular convex polyhedrons, but their efficiency still needs to be improved. Therefore, we first test our two packing algorithms on irregular convex polyhedrons. Experimental results show that our algorithm has high packing density and efficiency in the bin packing problem and the open dimension problem for convex polyhedrons, as shown in Fig. 6. The second type of test data involves 3D printing parts, comprising test data of bar-shaped, ring-shaped, and mixed parts from the Thingi10K dataset [29]. In these tests, our algorithm still achieves good packing density for both single shape and mixed shape, as shown in Figs. 5, 7, and 8. The final test concerns the packing of complex triangular mesh models, such as the bunny model, which consists of over 5000 triangles. Fig. 13 shows the packing results of our algorithm for the open dimension problem and the bin packing problem, respectively. Overall, thanks to physical simulation allowing objects to adjust their positions and rotations, our method is able to achieve a higher packing density in these data. Moreover, through mesh convex decomposition and FFT-based collision metric, our method realizes efficient dynamics simulation and gap filling for complex models.

By making some modifications to the basic algorithmic framework, our algorithm can be applied to solve both the open dimension problem and the variant bin packing problem. Coupling it with the finite element simulation of soft bodies, we convert rigid bodies into soft bodies based on the rigid body packing results, as shown in Fig. 14. Since soft bodies deform under force, they can fill more gaps and

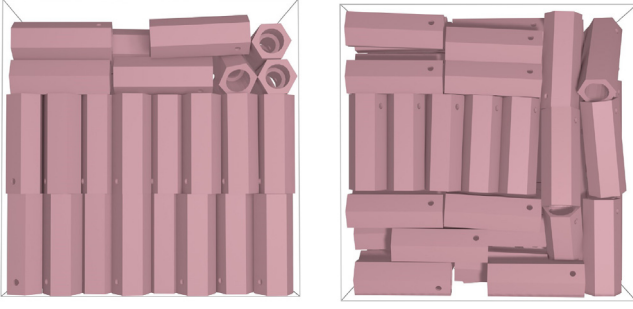


Fig. 7. Our algorithm packs 169 long hollow parts into a cube with a packing density of 35.88% within 78 s.

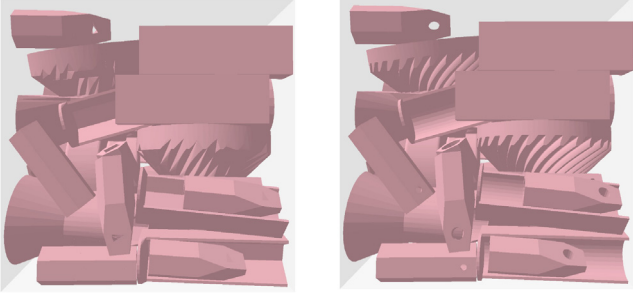


Fig. 8. Our packing results of mixed parts dataset, with the packing density of 42.95%, and 235 s. **Left:** Packing results for convex decomposition meshes. **Right:** The convex decomposition meshes are replaced by the origin meshes.

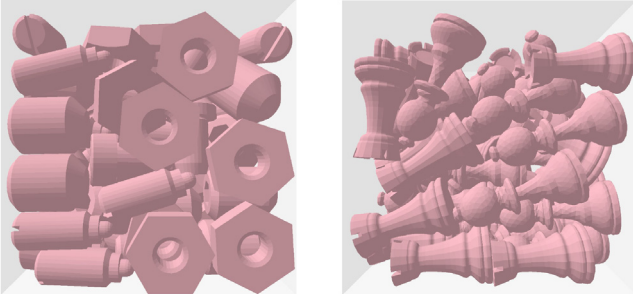


Fig. 9. **Left:** Our algorithm packs 95 tools into a box with a packing density of 48.91%, and 113 s. **Right:** Our algorithm packs 64 chess pieces into a box with a packing density of 36.26%, and 78 s.

achieve better packing results. In the context of e-commerce goods packaging, considering the deformation of soft bodies during packing can provide a more accurate reference for selecting the appropriate cardboard boxes for packaging. Considering low-dimensional cases, our algorithm, with minor modifications, can also achieve the packing of 2D irregular objects, as shown in Fig. 15. For complex 2D polygons from the dental dataset [30], we first perform a conservative polygon simplification for stable dynamics simulations. Then, the packing is done using a similar pipeline as in 3D.

Scalability for the number of shapes. To test the scalability of the algorithm, we packed different quantities of mushroom models and recorded the time required for packing. As shown in Fig. 11, as the quantity increases, the time required to compute the minimum container height to accommodate all the models increases. In addition, we packed nearly 400 dental models within 250 s, reaching a packing density of 50.05%. In the 3D dental dataset, each dental model on average is decomposed into 9 convex meshes, meaning our algorithm can still efficiently achieve packing within the range of 3600 convex meshes. If more frequent container shaking is allowed, the packing

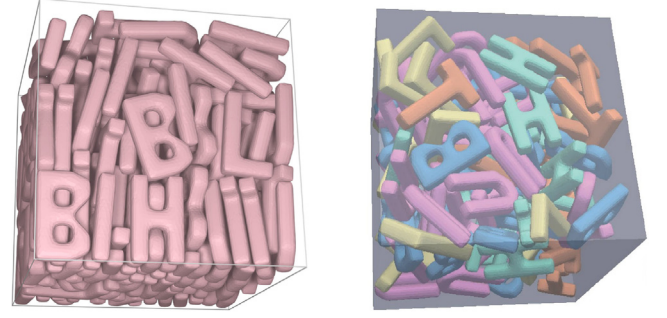


Fig. 10. **Left:** Our algorithm packs 204 letters into a cube container with a packing density of 46.59%, and 376 s. **Right:** Ma et al. [14] packs 104 letters into a cube container with a packing density of 26.2%, and 2880s.

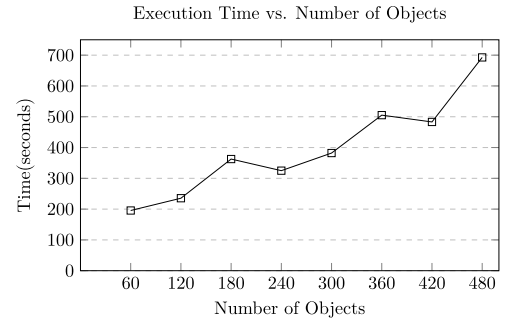


Fig. 11. Execution Time vs. Number of Objects for mushroom model packing.

density can reach 55.56%. As shown in Fig. 16, the container shakes at each blue node in the line chart, with object insertion occurring between these nodes. Each shake of the container takes about 2 seconds, and as the packing density increases, more shakes are needed to insert new objects.

The effect of shaking parameter. In order to analyze the effect of shaking parameters and shaking modes on the packing object arrangement, we conducted two sets of experiments according to shaking direction. With the same initialization configuration, the container vibrates only along the X -axis in the first set of experiments, and the container vibrates alternately along the X -axis and the Y -axis in the other set of experiments. Fig. 12 shows the height of the container holding all the objects under different vibration parameters after 10 oscillations. Because large amplitude and vibration frequency may cause the object to penetrate, we control the amplitude within $0.2S_x$, and $\omega \approx 2\pi$ (rad/s) for the experiments. The experimental results show that more shaking directions and larger amplitudes can obtain better results at the same angular frequency.

Comparisons. We compare the packing performance of our algorithm with 3 recent works using the same polyhedrons. The comparison results are shown in Table 1. Romanova et al. [1] use 42,950 s to pack 80 polyhedra with a packing density of 53.66%. Cui et al. [11] use 329 s to pack 80 polyhedrons inside a cube of size $36.98 \times 41.98 \times 56.37$, and the packing density is 51.27%. We optimized the packing of identical 80 polyhedrons to minimize the container height. Our algorithm achieves a packing density of 54.95% in 284 s. Within 102 s, our algorithm is able to achieve a packing density exceeding 51.27%. When the container size is $36.98 \times 41.98 \times 56.37$, 86 polyhedrons are packed by our method, and the packing density reaches 54.47%.

We pack the triangular mesh models of tools, letters, and chess pieces which are from the paper of Ma et al. [14]. Their approach packs 77 tools with a packing density 47.5% and a computing time of 33 min. For the chess pieces, their method packed 60 models with a packing density of 32.5% in 40 min. Our method achieves a higher packing density in just two minutes, as shown in Fig. 9. The packing results for

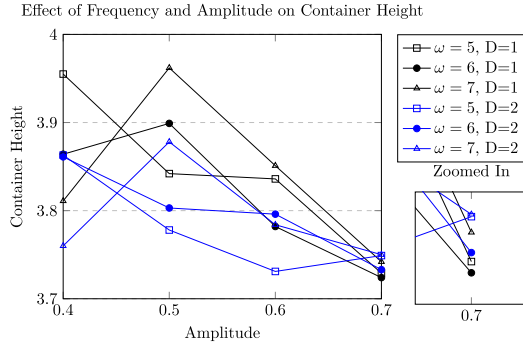


Fig. 12. Container Height vs Amplitude for Different Frequencies. $D = 1$ means that the container shakes only along the X -axis direction, and $D = 2$ means that the container shakes alternately along the X -axis and the Y -axis.

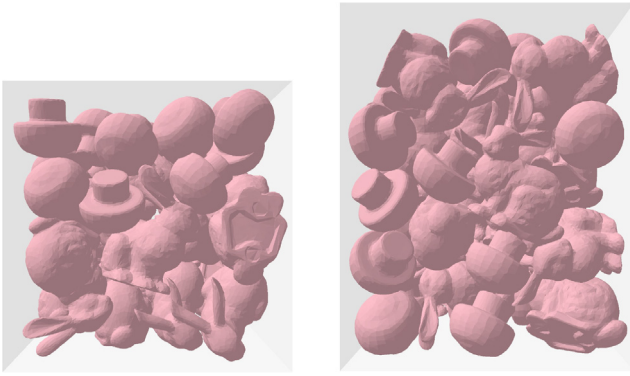


Fig. 13. **Left:** The minimum height of the container to pack 20 bunnies and 20 mushrooms is 4.01, with a packing density of 42.127% within 105 s. **Right:** 22 bunnies and 38 mushrooms are packed into the container of given size $4 \times 4 \times 5$, with a packing density of 44.849% within 127 s. When objects are allowed to be scaled and inserted, the packing density reaches 47.702%.

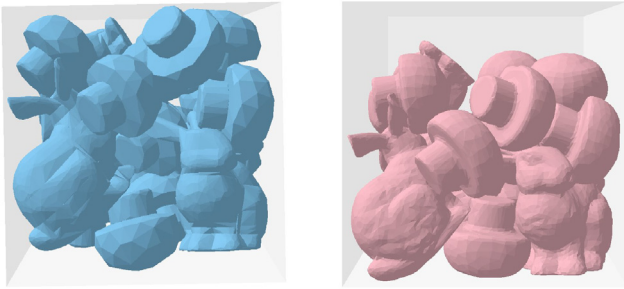


Fig. 14. Soft body packing. On the result of rigid body packing(left), soft body simulation is used to improve packing density (right).

the letters are shown in Fig. 10. Table 2 demonstrates that our method outperforms their method in terms of packing density and efficiency. In our methods, initialization contributes significantly to the packing density and is highly efficient. When initialization results are poor, the contribution during the optimization phase is high. We also compare our method with the methods of Xue et al. [30] and Xatlas [31] on the 2D open dimension problem. For the 2D dental dataset, the method of Xue et al. has lower packing density and efficiency than Xatlas packer [30]. The time consumption of our method is higher than Xatlas, but our packing result is better.

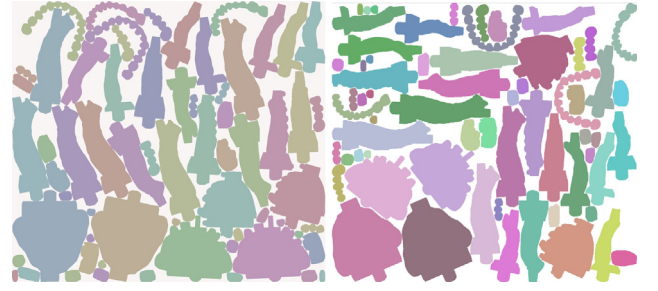


Fig. 15. 2D irregular packing. **Left:** 64 objects from dental model dataset [30] are packed into a rectangle by our algorithm, with the packing density of 71.17%. **Right:** Xatlas packs the same models using the brute force option with a packing density of 70.11%.

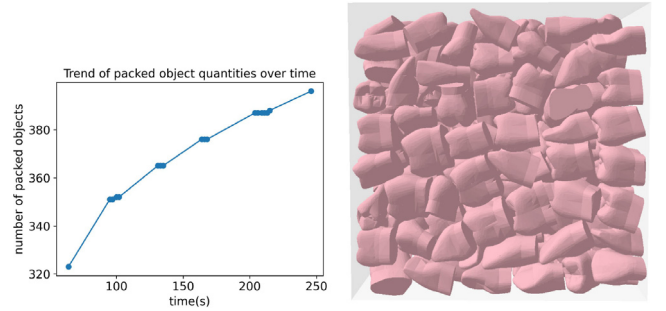


Fig. 16. **Left:** Line chart of the number of packed teeth and time. **Right:** After completing initialization at 64 s, the packing density of 50.05% is reached at 246 s.

Table 1

Comparison of time consumption and packing density between our methods and other packing methods in polyhedron packing. Here, BPP refers to the variant bin packing problem and ODP refers to the open dimension problem.

Methods	Number	Time(s)	Density(%)
Romanova et al. [1]	80	42950	53.66
Ma et al. [14]	77	2700	51.3
Cui et al. [11]	80	329	51.27
Ours (BPP)	86	277	54.47
Ours (ODP)	80	102	51.27
Ours (ODP)	80	284	54.95

6. Conclusion and future work

We propose a versatile packing framework based on physical simulation, which is suitable for diverse data and different packing problems. Inspired by researches on granular materials, and using dynamics simulation, our method achieves efficient packing of hundreds of irregular objects in a container to move and rotate, our algorithm achieves a high packing density. We also explore the applicability of this algorithm to soft body packaging. Based on the results of rigid body packing, the soft body simulation results show that the packing density is higher than that of rigid body packing.

Our approach, which relies on the stability of rigid body dynamics simulations, has two limitations. First, as the number of rigid bodies increases, the physics simulation takes longer. Therefore, our method is not currently suitable for packing thousands of 3D irregular objects. In future work, we will consider keeping the partially packed objects and containers relatively static to improve the efficiency of packing, while using batch packing to solve the problem of large-scale packing. Secondly, we ensure that all rigid bodies in the results of the dynamics simulation do not intersect by placing a collision envelope on the object surface. The collision envelope is the addition of a thin peripheral

Table 2

Comparisons of time consumption and packing density between our method and method of Ma et al. on different datasets.

Dataset	Ma et al. [14]		Initialization(Ours)		Optimization(Ours)		Ours	
	Density (%)	Time (s)	Density (%)	Time (s)	Density (%)	Time (s)	Density (%)	Time (s)
Tools	47.5	1980	44.54	77	4.37	36	48.91	113
Chess pieces	32.5	2400	28.94	31	7.32	47	36.26	78
Letters	26.2	2880	44.68	117	1.91	259	46.59	376

region to the surface of the object. It increases the gaps between objects, resulting in reduced space utilization of the packing result. A promising direction is to improve the rigid body dynamics simulation method for the packing problem. We will also consider combining reinforcement learning to determine the packing order and improve placement strategy in the future.

CRedit authorship contribution statement

Qiubing Zhuang: Writing – review & editing, Writing – original draft, Methodology. **Zhonggui Chen:** Writing – review & editing, Methodology. **Keyu He:** Methodology. **Juan Cao:** Writing – review & editing. **Wenping Wang:** Writing – review & editing, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

The work was supported by the National Natural Science Foundation of China (Nos. 61972327, 62272402, 62372389), the Natural Science Foundation of Fujian Province, China (No. 2022J01001), and the Fundamental Research Funds for the Central Universities, China (No. 20720220037).

References

- [1] Romanova T, Bennell J, Stoyan Y, Pankratov A. Packing of concave polyhedra with continuous rotations using nonlinear optimisation. *European J Oper Res* 2018;268(1):37–53.
- [2] Bortfeldt A, Wäscher G. Constraints in container loading – A state-of-the-art review. *European J Oper Res* 2013;229(1):1–20.
- [3] Zhao X, Bennell JA, Bektaş T, Dowsland K. A comparative review of 3D container loading algorithms. *Int Trans Oper Res* 2016;23(1–2):287–320.
- [4] Wang F, Hauser K. Robot packing with known items and nondeterministic arrival order. *IEEE Trans Autom Sci Eng* 2021;18(4):1901–15.
- [5] Hu R, Xu J, Chen B, Gong M, Zhang H, Huang H. TAP-Net: Transport-and-pack using reinforcement learning. *ACM Trans Graph* 2020;39(6):232:1–232:15.
- [6] Zhao H, Pan Z, Yu Y, Xu K. Learning physically realizable skills for online packing of general 3D shapes. *ACM Trans Graph* 2023;42(5):165:1–21.
- [7] Zhao H, Zhu C, Xu X, Huang H, Xu K. Learning practically feasible policies for online 3D bin packing. *Sci China Inf Sci* 2022;65(1):112105.
- [8] Zhao H, Yu Y, Xu K. Learning efficient online 3D bin packing on packing configuration trees. In: *International conference on learning representations*. 2021.
- [9] Vanek J, Galicia JAG, Benes B, Měch R, Carr N, Stava O, et al. PackMerger: A 3D print volume optimizer. *Comput Graph Forum* 2014;33(6):322–32.
- [10] Chen X, Zhang H, Lin J, Hu R, Lu L, Huang QX, et al. Dapper: decompose-and-pack for 3D printing. *ACM Trans Graph* 2015;34(6): 213–1.
- [11] Cui Q, Rong V, Chen D, Matusik W. Dense, interlocking-free and scalable spectral packing of generic 3D objects. *ACM Trans Graph* 2023;42(4):1–14.
- [12] Hu H, Zhang X, Yan X, Wang L, Xu Y. Solving a new 3D bin packing problem with deep reinforcement learning method. 2017, arXiv preprint arXiv:1708.05930.
- [13] Lamas-Fernandez C, Bennell JA, Martinez-Sykora A. Voxel-based solution approaches to the three-dimensional irregular packing problem. *Oper Res* 2023;71(4):1298–317.
- [14] Ma Y, Chen Z, Hu W, Wang W. Packing irregular objects in 3D space via hybrid optimization. *Comput Graph Forum* 2018;37(5):49–59.
- [15] Bernal JD, Mason J. Packing of spheres: Co-ordination of randomly packed spheres. *Nature* 1960;188(4754):910–1.
- [16] Radin C. Random close packing of granular matter. *J Stat Phys* 2008;131(4):567–73.
- [17] Pouliquen O, Nicolas M, Weidman P. Crystallization of non-Brownian spheres under horizontal shaking. *Phys Rev Lett* 1997;79(19):3640.
- [18] Amirifar R, Dong K, Zeng Q, An X, Yu A. Effect of vibration mode on self-assembly of granular spheres under three-dimensional vibration. *Powder Technol* 2021;380:47–58.
- [19] Yu AB, An XZ, Zou RP, Yang RY, Kendall K. Self-assembly of particles for densest packing by mechanical vibration. *Phys Rev Lett* 2006;97(26):265501.
- [20] Gilbert EG, Johnson DW, Keerthi SS. A fast procedure for computing the distance between complex objects in three-dimensional space. *IEEE J Robot Autom* 1988;4(2):193–203.
- [21] Wei X, Liu M, Ling Z, Su H. Approximate convex decomposition for 3D meshes with collision-aware concavity and tree search. *ACM Trans Graph* 2022;41(4):1–18.
- [22] Wang L, Guo S, Chen S, Zhu W, Lim A. Two natural heuristics for 3D packing with practical loading constraints. In: *Proceedings of the 11th Pacific rim international conference on trends in artificial intelligence*. 2010, p. 256–67.
- [23] Gzara F, Elhedhli S, Yildiz BC. The pallet loading problem: Three-dimensional bin packing with practical constraints. *European J Oper Res* 2020;287(3):1062–74.
- [24] Wang F, Hauser K. Dense robotic packing of irregular and novel 3D objects. *IEEE Trans Robot* 2022;38(2):1160–73.
- [25] Zhao H, She Q, Zhu C, Yang Y, Xu K. Online 3D bin packing with constrained deep reinforcement learning. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021;vol. 35(1):741–9.
- [26] NVIDIA Corporation. cuFFT: Fast fourier transform library. 2023, <https://developer.nvidia.com/cufft>.
- [27] Catto E. Box2D: A 2D physics engine for games. 2023, <https://box2d.org>.
- [28] NVIDIA Corporation. NVIDIA PhysX SDK. 2023, <https://developer.nvidia.com/physx-sdk>.
- [29] Zhou Q, Jacobson A. Thingi10K: A dataset of 10,000 3D-printing models. 2016, arXiv preprint arXiv:1605.04797.
- [30] Xue T, Wu M, Lu L, Wang H, Dong H, Chen B. Learning gradient fields for scalable and generalizable irregular packing. In: *SIGGRAPH Asia 2023 conference papers*. 2023, p. 1–11.
- [31] Young J. Xatlas. 2024, <https://github.com/jpcy/xatlas>.